

Assignment no.5

Title: Imagine you are developing a system to manage student grades in a school. The grades are stored in a matrix where rows represent students, and columns represent subjects. However, most students do not take all subjects, resulting in a sparse matrix where most elements are zero. Given the sparse matrix representing student grades below, implement a solution using arrays to efficiently manage and manipulate the grades data: Perform operations such as calculating the average grade for each subject, identifying students with the highest grades, and finding the subject with the highest average grade.

Name: Diya Ravindra Bhansali

PRN: 125B1F271

Class: SY BTech IT

Source Code:

```
/*
```

```
Assignment No: 05
```

```
Name: Diya Ravindra Bhansali
```

```
PRN: 125B1F271
```

```
Class & Div : SY BTech IT A4
```

```
*/
```

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
const int subjCount = 4;
```

```
const int studCount = 4;
```

```
float studMatrix[][4] =
```

```
{{0, 0, 0, 78},{0, 88, 0, 0},{0, 0, 89, 0},{0, 0, 0, 88}};
```

```
struct stude_marks
```

```

{
    int row, col;
    float val;
};
stude_marks students[studCount*subjCount];
int stude_index=0;
void convert_sparse()
{
    int i,j;
    for(i=0;i<studCount;i++)
    {
        for(j=0;j<subjCount;j++)
        {
            if(studMatrix[i][j]!=0)
            {
                students[stude_index].row = i + 1;
                students[stude_index].col = j + 1;
                students[stude_index].val = studMatrix[i][j];
                stude_index++;
            }
        }
    }
}
void print_sparse()
{
    cout << "\nSparse Matrix\n";
    cout << "Row\tCol\tGrade\n";

```

```

for(int i = 0; i < stude_index; i++)
{
    cout << students[i].row << "\t"<< students[i].col << "\t"<<
students[i].val << endl;
}
}

void calculate_average()
{
    float total[subCount] = {0};
    int count[subCount] = {0};
    for(int i = 0; i < stude_index; i++)
    {
        int subject = students[i].col - 1;
        total[subject] += students[i].val;
        count[subject]++;
    }
    cout << "\nAverage Grade of Each Subject\n";
    for(int i = 0; i < subCount; i++)
    {
        if(count[i] != 0)
        {
            cout << "Subject " << i + 1<< " : " << total[i] / count[i] << endl;
        }
        else
        {
            cout << "Subject " << i + 1<< " : 0" << endl;
        }
    }
}

```

```

    }
}
void highest_grade()
{
    float highest = students[0].val;
    for(int i = 1; i < stude_index; i++)
    {
        if(students[i].val > highest)
        {
            highest = students[i].val;
        }
    }
    cout << "\nStudent with Highest Grade\n";
    for(int i = 0; i < stude_index; i++)
    {
        if(students[i].val == highest)
        {
            cout << "Student " << students[i].row<< " -> Subject " <<
students[i].col<< " -> Grade " << students[i].val << endl;
        }
    }
}

```

```

void highest_average_subject()
{
    float total[subjCount] = {0};
    int count[subjCount] = {0};

```

```

for(int i = 0; i < stude_index; i++)
{
    int subject = students[i].col - 1;
    total[subject] += students[i].val;
    count[subject]++;
}
float highestAverage = -1;
int highestSubject = -1;
for(int i = 0; i < subjCount; i++)
{
    if(count[i] != 0)
    {
        float average = total[i] / count[i];
        if(average > highestAverage)
        {
            highestAverage = average;
            highestSubject = i;
        }
    }
}
cout << "\nSubject with Highest Average Grade\n";
cout << "Subject " << highestSubject + 1 << " -> Average = " <<
highestAverage << endl;
}

int main()
{

```

```

int choice;
convert_sparse();
do
{
    cout << "\n=====";
    cout << "\n    STUDENT GRADE SYSTEM";
    cout << "\n=====";
    cout << "\n1. Display Sparse Matrix";
    cout << "\n2. Calculate Average Grade";
    cout << "\n3. Find Student with Highest Grade";
    cout << "\n4. Find Subject with Highest Average";
    cout << "\n5. Exit";
    cout << "\n\nEnter your choice: ";
    cin >> choice;

    switch(choice)
    {
        case 1:
            print_sparse();
            break;
        case 2:
            calculate_average();
            break;
        case 3:
            highest_grade();
            break;
        case 4:

```

```

        highest_average_subject();
        break;
    case 5:
        exit(0);
    default:
        cout << "\nInvalid choice!";
    }
} while(choice != 5);
return 0;
}

```

Output:

```
/*
```

```
=====
```

STUDENT GRADE SYSTEM

```
=====
```

1. Display Sparse Matrix
2. Calculate Average Grade
3. Find Student with Highest Grade
4. Find Subject with Highest Average
5. Exit

Enter your choice: 1

Sparse Matrix

Row	Col	Grade
-----	-----	-------

1	4	78
---	---	----

2	2	88
---	---	----

3	3	89
4	4	88

=====

STUDENT GRADE SYSTEM

=====

1. Display Sparse Matrix
2. Calculate Average Grade
3. Find Student with Highest Grade
4. Find Subject with Highest Average
5. Exit

Enter your choice: 2

Average Grade of Each Subject

Subject 1 : No grades available

Subject 2 : 88

Subject 3 : 89

Subject 4 : 83

=====

STUDENT GRADE SYSTEM

=====

1. Display Sparse Matrix
2. Calculate Average Grade
3. Find Student with Highest Grade
4. Find Subject with Highest Average

5. Exit

Enter your choice: 3

Student(s) with Highest Grade

Student 3 -> Subject 3 -> Grade 89

=====

STUDENT GRADE SYSTEM

=====

1. Display Sparse Matrix
2. Calculate Average Grade
3. Find Student with Highest Grade
4. Find Subject with Highest Average
5. Exit

Enter your choice: 4

Subject with Highest Average Grade

Subject 3 -> Average = 89

=====

STUDENT GRADE SYSTEM

=====

1. Display Sparse Matrix
2. Calculate Average Grade
3. Find Student with Highest Grade

4. Find Subject with Highest Average

5. Exit

Enter your choice: 5

Process returned 0 (0x0) execution time : 46.306 s

Press any key to continue.

*/

Code Analysis Report

Sparse Matrix for Student Grades

Field	Detail
Student	Diya Ravindra Bhansali
PRN	125B1F271
Class	SY B.Tech IT, A4
Assignment No.	05
Language / Paradigm	C++ (procedural, with a plain struct)
Core Data Structure	Sparse matrix stored as a triplet array (row, col, value)

1. Overview

The program stores a 4x4 grade matrix where most entries are 0 (a student who hasn't taken / hasn't been graded in a subject). Rather than scanning the full dense matrix repeatedly, `convert_sparse()` walks it once and records only the non-zero entries as (row, col, val) triplets in a global `students[]` array, tracked by a running count `stude_index`. All later operations — printing, computing per-subject averages, finding the highest grade, and finding the subject with the highest average — work off this compact triplet list instead of the original 4x4 array.

This is the standard triplet (coordinate list / COO) representation of a sparse matrix, and it is applied correctly: `convert_sparse()` builds it once in `main()` and every other function reuses it. The sample run in the trailing comment confirms the four menu operations produce correct results for the given data.

2. Structural Analysis

2.1 Components

- `stude_marks` — a plain struct holding one non-zero entry: row, col, val.

- `students[] / stude_index` — a global fixed-size array acting as the sparse triplet list, plus a counter for how many slots are filled.
- `convert_sparse()`, `print_sparse()`, `calculate_average()`, `highest_grade()`, `highest_average_subject()` — free functions that all read the same global state rather than being organized behind a class.

2.2 Time Complexity

Operation	Complexity	Notes
<code>convert_sparse()</code>	$O(\text{rows} \times \text{cols})$	One-time pass over the full dense matrix ($4 \times 4 = 16$ checks here).
<code>print_sparse()</code>	$O(k)$	k = number of non-zero entries (<code>stude_index</code>).
<code>calculate_average()</code>	$O(k)$	One pass accumulating per-subject totals/counts.
<code>highest_grade()</code>	$O(k)$	Two passes over the triplet list: find max, then collect all matches.
<code>highest_average_subject()</code>	$O(k + \text{subjCount})$	Recomputes the same per-subject totals as <code>calculate_average()</code> , then scans <code>subjCount</code> averages.

All four query operations are linear in the number of non-zero entries rather than the full `studCount` x `subjCount` grid, which is exactly the payoff a sparse representation is meant to deliver.

3. Issues Identified

3.1 Correctness / Robustness

- `highest_grade()` initializes `highest = students[0].val` without checking `stude_index > 0` first. If the matrix were entirely zero (no grades entered at all), `stude_index` would be 0 and `students[0]` would be read before it was ever written — undefined behavior, not just a logic error.
- The trailing sample-output comment shows "Subject 1 : No grades available" for the subject with zero entries, but the actual code in `calculate_average()` prints "Subject 1 : 0" in that case. The recorded output no longer matches what this version of the code produces, which suggests the output block wasn't re-captured after the message text was changed.
- The menu's `cin >> choice` is read with no validation, same risk as in other assignments: a non-numeric entry leaves `cin` in a fail state and the loop will not behave sensibly.

3.2 Design

- The "sparse" storage doesn't actually save space here: `students[]` is declared with a fixed size of `studCount * subjCount`, i.e. the same capacity as the dense matrix it's replacing. The memory saving that a triplet list is normally used for only shows up if the array is sized (or grown) to the actual number of non-zero entries, not the worst case.
- `calculate_average()` and `highest_average_subject()` both build identical `total[]/count[]` per-subject arrays from scratch — the second function duplicates the first's logic instead of reusing it.

- All state (`studMatrix`, `students`, `stude_index`) is file-scope global rather than owned by a class or passed as parameters, so every function implicitly depends on shared mutable state. That makes the functions harder to test in isolation and easy to misuse in the wrong order (e.g. calling any query function before `convert_sparse()` runs).
- The grade matrix is hardcoded at compile time (`studMatrix[][4] = {...}`); there is no way for a user to enter their own grades interactively, which limits the program to the one built-in dataset.
- using namespace `std`; at file scope — minor, but worth avoiding as the codebase grows.

4. Suggested Optimizations

4.1 Fix the Empty-Matrix Edge Case

- In `highest_grade()`, check if (`stude_index == 0`) and print a "No grades recorded" message before touching `students[0]`, the same way `calculate_average()` already guards each subject's `count[i] != 0`.

4.2 True Sparse Storage

- Replace the fixed-size `students[studCount*subjCount]` array with `std::vector<stude_marks>` and `push_back()` the non-zero entries in `convert_sparse()`. This makes the container's size proportional to `k` (the actual number of non-zero grades) instead of the full dense matrix size, which is the actual point of a sparse representation.

4.3 Remove Duplicated Logic

- Factor the `total[]/count[]` accumulation shared by `calculate_average()` and `highest_average_subject()` into one helper, e.g. `void build_subject_stats(float total[], int count[])`, and have both functions call it. This removes the duplicated loop and keeps the two calculations consistent if the logic ever changes.

4.4 Encapsulation

- Wrap `studMatrix`, `students`, and `stude_index` inside a `SparseMatrix` class with methods like `fromDense()`, `print()`, `averagePerSubject()`, `highestGrade()`, and `highestAverageSubject()`. This removes the reliance on global state, makes the functions independently testable, and keeps `convert_sparse()` from needing to run before every other call by construction (e.g., run it in the constructor).

4.5 Input & Output Quality

- Allow grades to be entered interactively or read from a file, rather than hardcoded, so the program works for more than the one built-in matrix.
- Reconcile the "0" vs. "No grades available" message so the code and any recorded sample output stay consistent, and apply the same wording in both `calculate_average()` and any future functions with the same empty-subject case.
- Validate the menu choice input the same way as recommended for other console menu programs — check `cin.fail()`, then `cin.clear()` and `cin.ignore()` before re-prompting.

5. Conclusion

The core idea — convert a mostly-empty grade matrix into a compact (row, col, value) triplet list once, then run every query against that list instead of the full grid — is implemented correctly, and the linear-in-`k` complexity of `print_sparse()`, `calculate_average()`, `highest_grade()`, and `highest_average_subject()` shows the sparse representation being used as intended. The main gaps are a real edge-case bug in `highest_grade()` for an all-zero

matrix, an output message that no longer matches the code, storage that is only sparse in concept (the array is sized like the dense matrix), and repeated per-subject aggregation logic across two functions. None of these affect the correctness of the given sample run, but fixing the empty-matrix guard and switching `students[]` to a dynamically sized container would make the program correct in the general case and genuinely space-efficient.